

1. Liu's Routing Rule

Jason Liu proposes a simple hierarchy for Codex computer interaction: if a plugin can finish the job, use it. If a screen tool is necessary, choose the narrowest one that can finish the job. His three screen surfaces are Browser, Chrome, and Computer Use.

The rule is valuable because it connects tool choice to the state required by the task rather than to novelty.

2. Plugins First

A structured integration can expose stable identifiers, schemas, and actions directly. This reduces perception errors and can be much faster than navigating a user interface. A GitHub tool that accepts a repository and issue number is preferable to visually hunting through tabs when the desired operation is already represented.

But plugin availability does not imply adequate scope. If the integration cannot perform the last required step, a broader tool may still be necessary.

3. Browser

The in-app browser is well suited to public pages, rendered artifacts, and visual annotation. It provides an inspectable surface without inheriting the user's full browser profile. For local web development it can close the loop between code and rendered result.

Its strongest use is collaborative review: the user and agent can point at the same page state.

4. Chrome

Chrome becomes appropriate when the task depends on authentication, cookies, existing tabs, extensions, or profile-specific state. This is more capability and more private context. Use it because the state is required, not merely because the task happens on a website.

A public documentation lookup should normally remain in a public browsing surface. A signed-in internal dashboard may need Chrome.

5. Computer Use

Computer Use is the broadest path. It can operate native applications and bridge gaps between tools. Liu describes it as powerful and usually slower because the agent must observe, click, wait, and observe again. Kundel's Astra work adds another reason to use it: domain applications such as Blender can provide high-value inspection and editing environments.

Broadest does not mean best. Each additional degree of freedom adds possible failure modes.

6. Cross-Tool Handoffs

A good workflow may use multiple surfaces sequentially while keeping each substep narrow. Example:

1. use a connector to retrieve structured feedback;
2. edit code with shell and repository tools;
3. open the result in Browser for visual inspection;
4. if final delivery requires a native app unsupported by a plugin, use Computer Use for that last mile;
5. stop before a consequential send or publish action unless authorized.

This is tool composition, not tool maximalism.

7. Put Stable Routing in Guidance

Liu stores his routing rule in AGENTS.md so he does not have to restate it. This is a good use of durable guidance because the preference recurs across tasks and its conditions are legible. It also includes a boundary: research and drafting can proceed, while sending, publishing, purchasing, submitting, or changing account settings requires confirmation.

A durable routing rule should remain short enough that the agent can apply it before every task.

8. A Counterexample

Suppose a plugin can technically upload a file but cannot preview the transformed artifact. The narrowest single tool might be the plugin, yet the safest workflow may combine a local preview or Browser inspection before upload. The routing rule should optimize the whole task, including verification, not only the final side effect.

9. General Principle

Choose the representation that makes the relevant state explicit, the required action reliable, and the verification observable. Escalate to broader tools only when narrower ones cannot carry the requirement. Preserve human approval for consequential boundaries. This converts an expanding toolset into a comprehensible operating system.

References: Jason Liu, "Three Ways Codex Can Use a Computer"; OpenAI, Codex computer use and browser documentation; OpenAI, Codex customization.

10. Worked Routing Cases

Case A: summarize a public API announcement. Use public web or Browser; signed-in Chrome adds no necessary state. Case B: update a draft in an authenticated CMS but do not publish. Chrome may be appropriate because the draft and session are in the user's profile; the publish boundary remains closed. Case C: export a Blender scene after checking geometry. Computer Use or a dedicated Blender integration is justified because the native application represents the artifact. Case D: create a GitHub issue when a GitHub connector can do so directly. Prefer the structured connector over screen navigation.

11. Latency, Observability, and Privacy

Broader screen tools often require more observe-act-observe cycles. They may expose more unrelated user state and produce less structured logs than a narrow API action. These are costs even when the task succeeds. Tool selection should therefore consider latency, observability, and data exposure alongside raw capability.

12. Mixed Workflows

Do not insist on one tool for an entire task. Research with a public browser, manipulate repository files with shell tools, inspect the result in Browser, and use Computer Use only for the native final step if necessary. The narrowest-tool principle applies at the subtask level.

13. Failure Escalation

If a narrow tool fails, diagnose why before escalating. A plugin may lack one permission, return stale data, or simply not expose the needed action. If the missing capability is genuine, move one step broader. If the problem is authentication or a transient error, Computer Use may merely hide the same failure behind clicks.

14. Persistent Rule

A good routing rule fits in a few lines because its purpose is to choose a surface, not prescribe every interaction. Preserve the stable distinctions: structured integration first, public page versus signed-in browser versus native desktop, and explicit approval at consequential boundaries. Let the task supply the rest.

Independent Open Course

<https://github.com/dicnunuz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 22: Jason Liu on Computer Tools

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>